

MotorX — Work Plan

3 members · 6 sprints from here · SRS v1.0, Group 23

Status

Check	Result
Repo state	Scaffold only — every module file is a stub, no logic yet
Sprint 0 (setup)	Done
Conflicting docs found	<code>team-work-plan.md</code> and <code>sprints.md</code> disagree on sprint order — this plan replaces both
Architecture decisions already in <code>docs/architecture.md</code>	Worker owns its own repos · <code>loadLocalUser</code> middleware exists · health routes named · image upload is backend-mediated multipart · Zod selected for validation

Action: delete or merge the old planning docs so the team works from one source.

Build order

(*diagram above*)

Phase	Sprints	Why this order
Core marketplace	1-2	Smallest usable product — auth, dealers, listings, structured search
Inventory + search	3-4	Hardest, most technical pieces — CSV/ETL, then semantic search
Notify + admin	5	Depends on everything above already existing
Ship	6	Integration, performance, deployment

Ownership

Member	Modules
M1	Auth & Users, Dealers, Marketplace
M2	Inventory (ETL)
M3	Search, Notifications, Administration

Shared types live in `packages/shared-contracts` — all three import from there, none redefine locally.

Sprint 1 — Auth, dealer onboarding, listings skeleton

	M1	M2	M3
Backend	Firestore auth chain: <code>verifyFirebaseToken</code> → <code>loadLocalUser</code> → <code>requireAuthenticated</code> → <code>requireRole</code>	<code>UploadJobs</code> / <code>RejectedRecords</code> schemas only	<code>Listings</code> schema + indexes
API	<code>users</code> / <code>dealers</code> collections, dealer approval flow (role granted only on approval)	CSV upload endpoint — accept + validate type only	<code>GET /listings</code> — list + pagination, no search yet
Frontend	Login page, <code>AuthProvider</code> , <code>RoleGuard</code> reads role from <code>/api/v1/users/me</code>	Upload page shell (picker + validation message)	Marketplace page, <code>ListingCard</code> , buyer portal shell

Do now, not later: split `buyer.routes.tsx` into `publicBuyerRoutes` (`/marketplace`, `/listings/:id`) VS `protectedBuyerRoutes` from the start.

Sprint 2 — Marketplace CRUD, structured search, health routes

	M1	M2	M3
Backend	Full listing CRUD + ownership checks	Link <code>UploadJobs</code> ↔ <code>Listings</code> via <code>sourceUploadJobId</code>	<code>GET /search</code> — all filters, sorting, pagination
API	Image upload: multipart → validate → S3/MinIO → <code>listings.images[]</code>	—	Health routes: <code>/health/live</code> , <code>/health/ready</code> , <code>/api/v1/admin/system-health</code>
Frontend	Dealer dashboard (listing count, basic stats)	—	Filter sidebar, sort controls, pagination UI

Sprint 3 — CSV upload → ETL core

M1	M2	M3
— Support role	Redis+BullMQ queue → worker → Extract → Normalize → Validate → exact-match dedup → save valid rows → store rejected rows	Support role
— —	Status tracking: Pending → Processing → Completed/CompletedWithErrors/Failed	—
— —	Upload history + rejected-records UI	—

Deferred to Sprint 4: categorization, enrichment, embeddings. Get CSV rows becoming listings first.

Sprint 4 — Embeddings, semantic + hybrid search

M1	M2	M3
— Support role	Categorize + enrich + generate embeddings — same model for listings and queries	NL query parser (rule-based, in-Node — not a hosted service)
— —	Retry failed jobs; refine dedup if time allows	Fuzzy matching + hybrid ranking (structured + fuzzy + semantic)
— —	—	NL search bar, no-results handling

Sprint 5 — Notifications + admin

M1	M2	M3
— Dashboard polish with real data	Wire worker's existing notification creation to real job events	Admin dashboard: users/dealers/listings/uploads
— —	—	Audit logs — start with dealer approval + user suspension only
— —	—	System stats (total listings, total dealers)

If time remains: add `savedSearches` here — not before.

Sprint 6 — Integration, testing, deployment

All members together.

Area	Target
Cross-integration	UC-01 (upload→listing→search), UC-02 (NL query→results), UC-03 (admin visibility)
Security check	Dealer A can't touch Dealer B's listing → 403
ETL edge case	Partial-failure batch splits correctly into listings vs. rejected records
Performance	Structured search <2s · semantic search <5s · 95% of API calls <2s · 5,000-record CSV <2 min
Ops	Docker Compose finalized, CI/CD, docs, deploy

Deferred or cuttable

Item	Call
<code>savedSearches</code>	Optional — Sprint 5+ only, if time allows
Saved listings (bookmarking)	Not in SRS — remove from any UI mockups now
Fuzzy dedup matching	Exact-match first; refine only if Sprint 3 finishes early
Full audit coverage	Two event types first (approval, suspension); expand later
Search relevance tuning	Get it working in Sprint 4; tune weights in Sprint 6 if time remains

This week

1. Delete or merge `sprints.md`
2. Confirm team is building against backend-mediated image upload (already decided)
3. Strip "saved listings" from any buyer UI mockups
4. Start Sprint 1